Computer Security Symposium 2026
19-23 October 2026

Applicability Evaluation of Greyhound, a State Transition-Oriented Directed Greybox Fuzzer, to Embedded Systems

Jin Koshino[1,a)] Yixiao Li[1,b)] Yutaka Matsubara[1] Hiroki Kawashima[2] Satoru Funahashi[2]
Hideaki Murata[2] Hiroaki Takada[1]

Abstract: Fuzzing embedded drone systems is challenging because both communication protocols and flight-control applications exhibit state-dependent behavior, making faults in deep states difficult to reach using random inputs. This study evaluates the applicability of Greyhound, a state-machine-guided directed greybox fuzzer, to embedded systems. We first conducted a replication experiment on a Raspberry Pi Wi-Fi client and confirmed that Greyhound could reproduce previously reported anomalous behaviors, while also identifying practical requirements for reproducibility and protocol extension. We then extended Greyhound with an MQTT state machine and applied it to an ESP32-based physical drone. The experiment detected protocol-related anomalies and implementation defects that affected communication and flight operation. These results demonstrate that Greyhound can be extended to test drone-control systems whose communication and application behavior depend on state transitions.

Keywords: drone cybersecurity, greybox fuzzing, stateful fuzzing, embedded systems, MQTT

1. Introduction

In recent years, several cyberattacks targeting the wireless communications of drones have been reported, resulting in drones being hijacked or captured. For example, in 2016, a high-performance drone deployed by government agencies and police forces was rendered uncontrollable from a distance of 2 km using equipment costing less than $40[1]. Also in 2016, a tool that exploits vulnerabilities in a wireless transmission control protocol to take control of a drone was reported[2]. These cyberattacks exploit software vulnerabilities as an initial attack vector. Therefore, identifying software vulnerabilities is essential for protecting drones against cyberattacks.

In this study, we propose a system that can efficiently detect software vulnerabilities in drones that communicate over wireless networks using MQTT. In particular, we employ fuzz testing, which has a proven track record in discovering previously unknown vulnerabilities.

Drone-control applications consist of multiple states for managing flight status. In addition, wireless communication establishes a connection through multiple stages defined by a protocol, and the appropriate packet must therefore be transmitted according to the current stage. For systems with these characteristics, simply providing random inputs to the target is insufficient for efficient and comprehensive testing.

We therefore implement fuzz testing for drone systems by extending "Greyhound," an existing method that interprets a communication process as a state machine and enables efficient testing of deep states. In addition to the wireless communication protocol used to communicate with the drone, we model the application program that defines the drone's behavior as a state machine, thereby attempting to induce and detect anomalous behavior caused by malformed packets.

The contributions of this study are as follows:

- We validated Greyhound's operation and conducted a replication experiment to evaluate it as a fuzzing method for stateful systems.
- We identified the knowledge, configuration requirements, and practical caveats necessary to use Greyhound that are not explicitly described in the original paper.
- As a case study, we extended Greyhound with an MQTT state machine and conducted experiments on a drone-control system, demonstrating its ability to detect implementation defects that affect both communication and flight operation.

The remainder of this paper is organized as follows. Section 2 presents related work on Greyhound and security testing of drone systems. Section 3 explains the mechanism by which Greyhound performs software testing. Section 4 summarizes practical considerations for using Greyhound by reproducing the detection results reported in the original paper. Section 5 extends Greyhound and applies it to security testing of a drone system. Section 6 concludes this paper.

[1] Graduate School of Informatics, Nagoya University
[2] Research & Innovation Center at Mitsubishi Heavy Industries, Ltd.
[a)] j_koshino@ertl.jp
[b)] liyixiao@ertl.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

2. Related Work

2.1 Recent Developments Related to Greyhound

Greyhound aims to provide a research baseline for the systematic testing of wireless communication protocol implementations. Since Greyhound was released, several studies have proposed extensions and improvements to the method.

Extension to additional target protocols. Greyhound has also been applied to communication protocols that it did not originally support. A testing system that coordinates multiple techniques, including Greyhound, has been proposed for Bluetooth Low Energy [3]. A framework for stateful fuzzing of edge devices that communicate over 5G has also been proposed [4]. By reusing Greyhound's state-machine and packet-generation infrastructure while generating test cases through combinatorial testing, the framework compensates for the limited input-space coverage of probabilistic fuzzing.

Efficient construction of state machines. Greyhound requires the communication protocol used by the target to be defined manually as a state machine. To address this limitation, subsequent studies have proposed methods for automatically constructing protocol state machines. U-Fuzz makes it possible to construct a state machine for an arbitrary IoT protocol solely from network-capture files [5], [6].

2.2 Security Testing of Drone Systems

Security testing of drones, and more broadly of robotic vehicles, has attracted growing attention, and a recent systematic literature review organizes the field along two axes: fuzzing the onboard control program and fuzzing the external communication surface [7]. On the control side, RVFuzzer was the first to target *input-validation bugs* that manifest as physical instability rather than as a memory-safety crash, mutating control parameters and using a control-instability metric as feedback [8]. PGFUZZ generalizes this idea by expressing safety and functional requirements as metric temporal-logic policies and steering mutation toward policy violations [9], while learning-guided approaches such as ICSEARCHER predict destabilizing *range specification bugs* without exhaustively invoking a simulator [10]. Complementarily, MAYDAY traces control-model instability back to the offending code in order to explain accidents after they occur [11]. These techniques are powerful, but they presuppose a control model or a high-fidelity simulator of the target vehicle, which is costly to construct for an arbitrary airframe.

A second line of work fuzzes the communication surface of drones and IoT devices. For drones specifically, the MAVLink ground-control protocol has been fuzzed to expose parsing and validation flaws [12]. For IoT devices in general, IoTFuzzer drives firmware through its companion mobile application so that protocol-valid messages can be generated without access to the firmware [13], and Snipuzz recovers coverage-like feedback in a black-box setting by inferring message snippets from device responses [14]. Closest to our target protocol, FUME models MQTT as Markov-chain state machines to fuzz brokers [15], and the more recent MBFuzzer exercises MQTT brokers under multi-party interactions [16]. These MQTT efforts, however, target broker (server) implementations running on commodity hosts, rather than a client embedded in a resource-constrained flight controller.

To the best of our knowledge, these directions have remained largely separate: drone fuzzing concentrates on the control program and its parameters and relies on models or simulators, while the communication-side efforts above target broker or server implementations on commodity hosts. A third, complementary direction—state-machine-guided protocol fuzzing, to which the tool used in this study belongs—drives the target through an explicit protocol state machine and validates each response against it, and has so far been demonstrated mainly on Wi-Fi, Bluetooth, and cellular stacks rather than on the application-level communication task of a flying drone. Our case study connects these directions by applying such a state-machine-guided, response-validating methodology to the MQTT communication task of a real ESP32-based drone, where a specification-noncompliant response or a mutated length field carries an immediate, physical consequence for the aircraft.

3. Mechanism of Greyhound

Greyhound is a greybox fuzzer designed to detect protocol noncompliance and implementation vulnerabilities by emulating the communication peer of the target under test. It communicates with the target and checks whether the actual response matches the response expected in the current communication state. This allows Greyhound to detect not only crashes but also behavior that does not comply with the protocol. To achieve this, Greyhound provides the following two functions:

- **Packet generation and parsing**
- **Protocol modeling as a state machine**

First, the packet-generation and parsing function is used by the machine running Greyhound to transmit packets in a specified format and infer the current connection state from response packets received from the target. Greyhound implements this function using `Scapy`, a Python module. If Scapy does not provide a packet definition for the protocol under test, a custom definition must be created. Greyhound includes its own packet definition for EAP-pwd.

Second, Greyhound defines a protocol as a state machine, representing each stage of the connection procedure as a state and interpreting received packets as transition triggers. Using the Python `transitions` module, Greyhound specifies, for each state, the packets transmitted during transitions, the expected response packets, the destination state selected according to each response, and timeout conditions. As a proof of concept, Greyhound defines state machines for both the access point (AP) and client (supplicant) sides of the WPA2 Wi-Fi authentication protocol. In this study, we define an MQTT state machine and confirm that Greyhound can communicate with a physical drone. Details are provided in Section 5.

Greyhound executes the connection procedure with the target according to the defined state machine. During this process, it attempts to induce anomalous behavior in the target by mutating selected field values in generated packets or by retransmitting a packet that was sent several seconds earlier. If the target

© 2026 Information Processing Society of Japan

Table 1 PC used to run Greyhound.

model	ThinkPad L15 Gen 4
CPU	Intel Core i7-1355U (10 Cores)
RAM	16GB
Storage	512GB NVMe SSD
OS	Ubuntu 18.04.6 LTS (5.4.0-150-generic)
wireless LAN dongle	Buffalo WLI-UC-AG300N

Table 2 Specifications of the Raspberry Pi target.

software version	Preinstalled	Modified
wpa_supplicant	2.8-devel	2.6
openssl	1.1.1c	1.0.1u
wpa_supplicant driver	nl80211	wext

crashes, it is expected to stop transmitting response packets. In other cases, the target software may fail to validate a mutated field as required by the protocol and may transmit a response in a situation in which it should not respond. Greyhound determines whether each received response packet is valid for the current state and records an invalid response as an `ANOMALY`, that is, evidence suggesting a potential vulnerability.

Greyhound has definitions of the protocol layers and fields required to generate packets, and it assigns mutation probabilities to the field values of packets transmitted in each state. A valid packet generated from the state machine is modified according to these mutation probabilities and then transmitted to the target device.

The initial mutation probabilities are statically configured for the protocol layers of packets transmitted in each state. To efficiently transmit mutated packets in deeper states, it is important to assign lower mutation probabilities to preceding states, thereby increasing the probability of reaching the target state. When testing for a known vulnerability, the vulnerability can be triggered more easily by assigning a higher mutation probability to the relevant protocol layer in the state in which the vulnerability may occur. When targeting other protocols or testing for unknown vulnerabilities, Greyhound can also dynamically adjust mutation probabilities according to execution results. One iteration is defined as the sequence from the start of connection authentication to disconnection and the subsequent transition back to the initial state. Greyhound evaluates the result of each iteration using a cost function and updates the mutation probabilities used in the next iteration. The cost function can be based on the number of states traversed, the interval between anomaly detections, the number of anomalies, or the time required for one iteration.

4. Replication Experiment with Greyhound

In this section, we conduct a replication experiment to determine whether Greyhound can detect the vulnerabilities reported in prior work. Table 1 lists the machine used to run Greyhound. As the target device, we used a Raspberry Pi 3 Model B Rev. 1.2 running Raspbian Buster. Based on publicly available information about the affected environment, we changed the versions of the software components and the driver used for EAP authentication from those preinstalled in the OS image. Table 2 compares the preinstalled and modified configurations.

Table 3 Results of the replication experiment.

Vulnerabilities / Inconsistencies	Greyhound [18]	This work
A1 CVE-2017-13077 (KRACK)	affected	Not affected
A2 CVE-2017-13078 (KRACK)	affected	Not affected
A3 CVE-2017-13080 (KRACK)	affected	Not affected
A4 CVE-2019-9499 (Dragonslayer)	affected	affected
A5 Invalid EAP version accepted	affected	affected
A6 Non-compliant handling of EAP-fail	affected	Not affected

4.1 Results of the Replication Experiment

As shown in Table 3, our experiment detected two of the anomalous behaviors reported in the original Greyhound paper.

The first was CVE-2019-9499 (**A4**). CVE-2019-9499 is a vulnerability in which the EAP-pwd implementation of the EAP peer in `wpa_supplicant` fails to validate the scalar and element values in EAP-pwd-Commit packets when it is built against a cryptographic library that does not provide explicit validation functions [17]. We were unable to detect this vulnerability using the software versions preinstalled in the operating system of the target device, even after conducting a run of 2,000 connection attempts. We therefore downgraded `wpa_supplicant` and OpenSSL and repeated the experiment. As a result, we confirmed that CVE-2019-9499 could be detected.

The second was a failure to reject invalid values in the EAP version field (**A5**). The original Greyhound study reports that this behavior is unlikely to pose a serious security risk to the tested client implementation, and our experiment confirmed that it also occurs with relatively recent versions of `wpa_supplicant` and OpenSSL.

However, we were unable to detect anomalous behavior caused by retransmitted packets in this experiment. KRACK (Key Reinstallation Attack), one of the vulnerabilities detected in the original Greyhound paper, is an attack that can enable the decryption or modification of WPA2-encrypted communications. During the four-way handshake performed at the end of WPA2 connection authentication, an attacker retransmits Message 3, causing the target device to reinstall the encryption key and reuse packet numbers and nonces. Depending on the cryptographic scheme and implementation, this may make it possible to decrypt or tamper with the communication. The reason we could not reproduce these vulnerabilities is that each vulnerability is detectable only with particular versions of `wpa_supplicant`, making it necessary to switch to the appropriate software version for each case. Although not all previously reported vulnerabilities were reproduced, these results demonstrate that the reconstructed Greyhound environment could reproduce and detect known vulnerabilities and protocol non-compliance.

4.2 Practical Considerations When Using Greyhound

Greyhound is an effective fuzzing method for testing bugs that may occur in deep states of stateful systems. However, the following points must be considered to ensure reproducibility and extensibility:

- Reproducing known vulnerabilities depends not only on the affected software versions but also on target-specific configurations that may not be documented in public vulnerability information. In our experiment, establishing the

© 2026 Information Processing Society of Japan

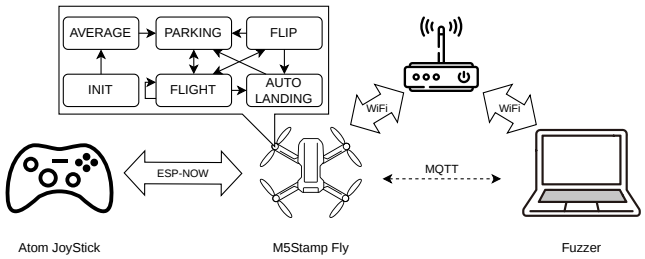


Fig. 1 Overview of the drone fuzzing environment.

EAP connection on the Raspberry Pi required configuring wpa_supplicant to use the wext driver, although this requirement was not described in the CVE entry. The driver setting was specific to the target environment rather than to the vulnerability itself. This experience shows that replication experiments must validate the complete software and driver configuration instead of relying solely on the CVE description.

- Custom packet definitions must be written accurately in accordance with the protocol specification. Greyhound uses Scapy to generate and parse packets. When the target protocol is not supported by Scapy, its packet structure must be defined manually. An incorrect definition therefore poses a threat to the validity of the evaluation results. Greyhound also requires its protocol state machine to be constructed manually. Recent studies such as U-Fuzz infer state machines from observed communication traces. Combining such observations with available protocol specifications may enable more accurate, target-specific state machine construction for Greyhound.

5. Case Study: Fuzzing a Drone System

The replication experiment in the previous section confirms that Greyhound is effective against Wi-Fi client implementations running on general-purpose platforms. To assess whether the same methodology generalizes beyond its original scope, we apply Greyhound to an embedded control system communicating over a protocol that was not part of the original proof of concept. Concretely, we build a demonstration environment in which Greyhound fuzzes the MQTT communication task of an open-source drone, and we discuss the anomalies detected during the experiment.

5.1 Targeting an Embedded Control System

As a target, we use **M5Stamp Fly**, an open-source drone development kit based on the ESP32 microcontroller. Figure 1 gives an overview of this target and the fuzzing environment we build around it: the M5Stamp Fly is normally driven by manual remote control over ESP-NOW, and we replace that link with an MQTT channel over WiFi so that Greyhound can act as the broker and fuzz the drone. The kit consists of the Stamp Fly airframe and an Atom JoyStick remote controller, and its firmware is built on top of the Arduino framework. The flight controller is implemented as a state machine whose main loop runs at 400 Hz and switches among six control modes: `INIT`, `AVERAGE`, `PARKING`, `FLIGHT`, `FLIP`, and `AUTO_LANDING`. Because the Arduino framework on the ESP32 is itself implemented as a FreeRTOS task, the firmware can be extended with additional preemptive tasks without being rewritten from scratch, which is a prerequisite for adding a communication task alongside the existing control loop.

By default, the airframe and the remote controller communicate through ESP-NOW [19], a lightweight connectionless protocol proprietary to the ESP32. ESP-NOW transmits data directly to a peer MAC address over the Wi-Fi physical channel without associating with an access point, and therefore never establishes the connection states that a stateful protocol exhibits. Since Greyhound drives fuzzing from a protocol state machine, a stateless protocol such as ESP-NOW is a poor fit. To obtain a stateful target that is also representative of realistic IoT deployments, we replace the control link with **MQTT**, a publish/subscribe protocol whose connection handshake and QoS message exchange constitute a well-defined state machine [20]. This substitution lets us evaluate two properties at once: whether Greyhound can be applied to a control system, and whether it can be extended to a protocol for which no state machine was previously defined.

5.2 Construction of the Verification Environment

Building the environment for this case study requires work on both ends of the control link. On the target side, we extend the M5Stamp Fly firmware with an MQTT communication task so that the drone can be driven over a stateful protocol instead of ESP-NOW. On the fuzzer side, we extend Greyhound with an MQTT state machine and application-level command handling so that it can act as the broker and steer the drone while mutating the traffic it sends. The remainder of this subsection describes these two components in turn.

5.2.1 Adding MQTT Task on the Drone

We extend the M5Stamp Fly firmware with a Wi-Fi/MQTT communication task that runs concurrently with the flight-control loop as a separate FreeRTOS task. Among the MQTT client libraries available for the PlatformIO ESP32 Arduino environment, we adopt ArduinoMqtt library (version 1.6.0), an Eclipse Paho-based implementation supporting MQTT 3.1.1, because it builds cleanly against the firmware's ESP32 SDK and provides a full protocol implementation. Both the MQTT task and the ESP-NOW link rely on Wi-Fi, so we initially enable them simultaneously using the `WIFI_AP_STA` mode of the ESP32, which allows the access-point and station interfaces to coexist on a single physical channel. In practice, however, running ESP-NOW (which transmits telemetry every 25 ms and receives control commands every 10 ms) together with periodic MQTT publish/subscribe processing destabilized both communication and flight. We therefore removed ESP-NOW and drove the drone over MQTT alone; relaxing the telemetry period to 1000 ms and the control period to 300 ms allowed communication and flight to be sustained together.

Communication between the drone (an MQTT client) and the host (an MQTT broker) uses two topics:

- **`drone/status`**: telemetry published by the drone, carrying a publish counter, the current control mode, the battery voltage, and the altitude, with fields separated by "`|`" (e.g., `694|PARKING_MODE|3.6v|0.04m`).

© 2026 Information Processing Society of Japan

- **drone/control**: flight commands published to the drone and the target of fuzzing. Its payload takes one of three forms: (1) `[heartbeat];` used only to keep the publish/subscribe channel alive (2) `[arm];` that requests a take-off/landing mode transition (3) `[config]r=0.1,p=0.0,y=-0.2,a=0.1;` that updates the roll, pitch, yaw, and altitude parameters.

5.2.2 Supporting MQTT State in Greyhound

On the fuzzer side, we extend an MQTT-capable build of Greyhound to model the MQTT connection and QoS 2 message exchange as a state machine and to interact with the drone at the application level. Figure 2 shows the resulting state machine. Starting from `INIT`, it brings up a session through `DISCONNECTED` and `RCV_CONNECT` to `CONNECTED`, which serves as the hub of the model: each MQTT packet received while connected (`PUBLISH`, `SUBSCRIBE`, `UNSUBSCRIBE`, `PINGREQ`) triggers a transition into the corresponding `RCV_*` state and returns to `CONNECTED` on the next `update`. The QoS 2 publish flow is modeled separately: a `broker_publish` drives the machine through `WAIT_PUBREC`, `RCV_PUBREC`, and `RCV_PUBREL` so that each control packet is interpreted as a distinct transition trigger, while a `receive_disconnect` or a `timeout_max` returns it to `DISCONNECTED`. On top of the protocol machine, we add handling for the two application topics: Greyhound parses incoming `drone/status` messages and logs the drone state, and it injects `drone/control` commands whose fields can be mutated by the fuzzer. To make the demonstration reproducible, we also add a keyboard-driven interface that lets the operator send control commands and enable fuzzing at arbitrary points during a run. With this setup, we confirmed on the physical drone that Greyhound and the M5Stamp Fly MQTT task communicate correctly and that the drone responds to the injected commands.

Preparing Greyhound for anomaly analysis also required two instrumentation fixes. First, the packet-capture logs were frequently incomplete because the log-saving routine silently swallowed every exception; the root cause was a field-name collision on `payload` between the TCP and MQTT layers in the Scapy-based packet construction, which we resolved by serializing the crafted packet into the TCP payload as raw bytes. Second, when a fuzzed packet caused the drone to reboot, the original Greyhound merely ended the iteration on a state timeout without recording an anomaly; we added a timeout callback that reports such timeouts as `CRASH` events together with the last fuzzed packet, so that reboot-inducing inputs are captured for later analysis.

5.3 Detected Anomalies

Fuzzing the MQTT communication task of the M5Stamp Fly demonstration produced several anomalies. Figure 3 shows a representative run: the upper part is Greyhound's state-transition log with the detected anomaly, and the lower part is the corresponding malformed `PUBLISH` packet inspected in Wireshark. We discuss the four most instructive anomalies below, together with the corrective actions they motivated.

Anomaly 1: unexpected UNSUBSCRIBE during the QoS 2 flow. Under MQTT QoS 2, after the broker sends `PUBREL` the client is expected to return `PUBCOMP`. We observed, however, that the drone sometimes sent `UNSUBSCRIBE` before `PUBCOMP`, which Greyhound flagged as an anomaly in the `RCV_PUBREC` state. Notably, this anomaly reproduced even with fuzzing disabled. Its direct cause is the quality of the wireless link: when the communication delay is large, `PUBREL` does not reach the drone within its 300 ms period. Placing the Wi-Fi access point, the drone, and the Greyhound host in the same room eliminated the anomaly. The underlying cause is that ArduinoMqtt's QoS 2 handling is too simplistic; a compliant client should wait for completion or a timeout before issuing `UNSUBSCRIBE` when a topic is still in the received (`PUBREC`-sent) state. This anomaly therefore reflects a robustness issue in the client implementation under degraded network conditions rather than in the protocol itself.

Anomaly 2: DISCONNECT in response to an invalid QoS value. When the QoS field of a `PUBLISH` packet was mutated, the client responded with a `DISCONNECT` packet and the MQTT connection was torn down; as a consequence, the airborne drone transitioned to the emergency `AUTO_LANDING` mode. According to the MQTT specification, a client that receives a `PUBLISH` with an invalid QoS value must close the connection, so this behavior is compliant and should *not* be reported as an anomaly. Treating every `DISCONNECT` as normal, on the other hand, would risk masking genuine anomalies. Moreover, the Scapy definition bundled with Greyhound represents `DISCONNECT` as a variant of the MQTT layer rather than as an independent layer, so it cannot be distinguished by the expected-layer check alone. We therefore implemented the `WAIT_PUBREC` → `DISCONNECTED` transition as a *conditional* transition: the guard treats an incoming `DISCONNECT` as expected (and thus not an anomaly) only when the preceding fuzzed `PUBLISH` carried a mismatched QoS value, and otherwise leaves the detection in place. After this change, mutating the QoS value no longer triggers a false anomaly and Greyhound correctly waits for the client to reconnect.

Anomaly 3: system crash caused by a mutated topic length. Mutating the length field of a `PUBLISH` packet caused a kernel panic on the M5Stamp Fly, rebooting the system and crashing the drone in flight. Analyzing the offending packet in Wireshark showed that the TCP payload and the MQTT message length were valid, but the topic length had been mutated to a very large value. Inspecting ArduinoMqtt revealed that `MQTTDeserialize_publish()` does validate the topic length—`readMQTTLenString()` checks it against the end of the message and returns a failure code when the topic does not fit (Figure 4). The check is nevertheless ineffective because the failure is never propagated: on entry the return value `rc` is set to the byte count returned by `MQTTPacket_decodeBuf()`, and the error path (`goto exit`) does not reset it to zero. Consequently, even when `readMQTTLenString()` fails, the function returns a nonzero (success) value, and the caller goes on to use a topic whose length field still holds the mutated, oversized value—an out-of-bounds access that crashes the firmware. We fixed it by no longer storing the return value of `MQTTPacket_decodeBuf()` in `rc` (using a separate variable for the byte count), so that `rc` stays zero until the parse fully succeeds; a failed topic-length check then correctly returns failure, after which the same mutation no longer crashes the firmware. We reported this bug upstream, and

© 2026 Information Processing Society of Japan

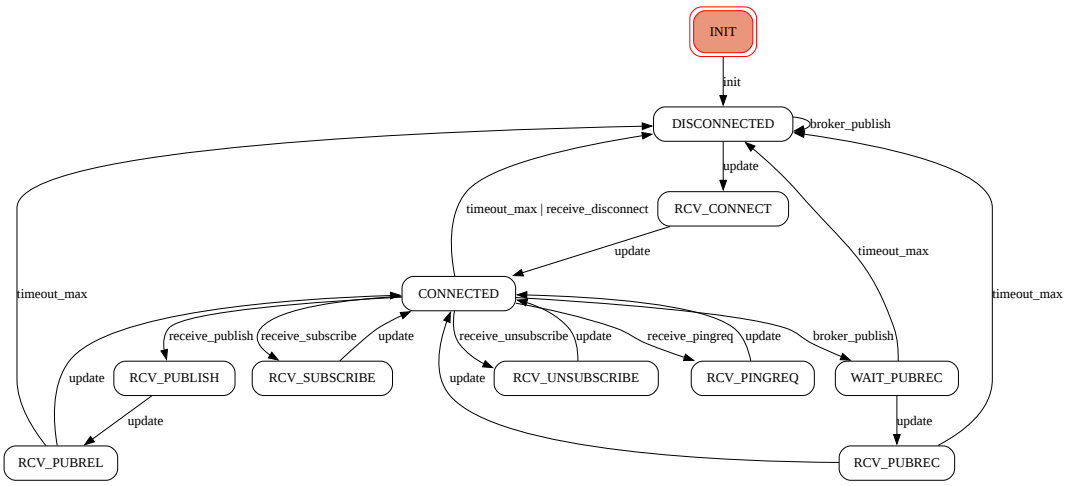


Fig. 2 Extending Greyhound by adding the MQTT state machine model.

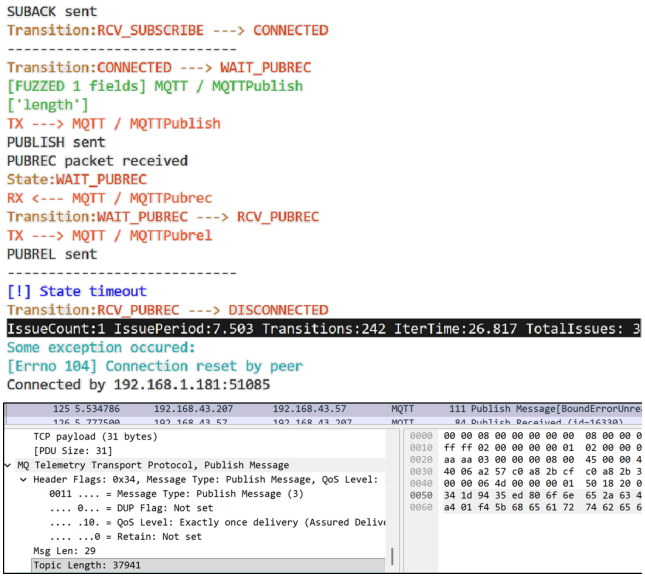


Fig. 3 Greyhound fuzzing the M5Stamp Fly MQTT task (an example screenshot of Anomaly 3).

```
int MQTTDeserialize_publish(...)
{
    ...
    int rc = 0;
    int mylen = 0;

    ...
    /* rc is set to the length-field byte count (> 0) */
    curdata += (rc=MQTTPacket_decodeBuf(curdata, &mylen));
    enddata = curdata + mylen;

    /* BUG: readMQTTLenString() does bounds-check the topic
       length, but rc is not reset on failure, so this path
       still returns the positive rc, i.e. success */
    if (!readMQTTLenString(topicName, &curdata, enddata) ||
        enddata - curdata < 0)
        goto exit;

    ...
    rc = 1;
exit:
    FUNC_EXIT_RC(rc);
    return rc;
}
```

Fig. 4 The topic-length handling bug in ArduinoMqtt causes Anomaly 3.

it has since been fixed [*1]. This anomaly illustrates that Greyhound, by driving the protocol into a deep state and mutating a length field there, can surface memory-safety defects that have concrete physical consequences for the drone.

Anomaly 4: loss of service due to the memory leak of an unreleased MQTT client buffer. After the topic-length fix, extended fuzzing runs eventually stopped receiving `PUBLISH` messages and repeatedly re-established bare `CONNECT` sessions. Because restarting Greyhound did not resolve the problem, we traced it to the drone side and found a memory leak in the MQTT client: ArduinoMqtt does not free the buffers it allocates when the `MqttClient` object is deleted. Declaring the relevant objects as `static` so that they are allocated only once resolved this instance. We nonetheless suspect that a memory leak may also exist within Greyhound itself: as the number of iterations grows, Greyhound becomes markedly slower (several seconds per log line) and eventually times out, a symptom that is cleared by restarting Greyhound but not by restarting the drone. Memory leaks are thus a plausible failure mode on both the target and the fuzzer, and bounding the memory footprint of long fuzzing campaigns remains future work.

[*1] `https://github.com/monstrenyatko/ArduinoMqtt/issues/44`

5.4 Discussion

This case study lets us draw several observations about applying Greyhound beyond its original scope. First, the four anomalies span a spectrum that a crash-oriented fuzzer would not report as a whole: a specification-compliant `DISCONNECT` that must not be flagged, a robustness weakness that surfaces only under a degraded wireless link, a memory-safety bug that crashes the drone, and a resource leak that degrades service over long runs. Because Greyhound reasons about the protocol state machine, it detects not only crashes but also unexpected packets, illegal state transitions, and flows that violate the specification, and can therefore uncover a broader range of defects than conventional fuzzers. Second, the anomalies also show where this breadth costs effort: distinguishing a compliant `DISCONNECT` from a genuine fault re-

© 2026 Information Processing Society of Japan

quired a conditional transition, and separating link-quality artifacts from implementation bugs required controlling the physical environment, so richer detection comes with a heavier burden of interpreting each report. Third, expressing the state machine and packet definitions with mainstream libraries—`transitions` and Scapy—kept the cost of targeting a new protocol and a new class of target modest, which in turn let us exercise the implementation more efficiently by sending packets tailored to each state. We caution, however, that this is a single demonstration on one target rather than a systematic evaluation; establishing how broadly the approach generalizes, and bounding the memory footprint of long fuzzing campaigns, remain future work.

6. Conclusion

This study investigated Greyhound, a state-machine-guided greybox fuzzer that emulates a communication peer and compares the actual response of the target with the response expected in each protocol state. We first conducted a replication experiment against a Wi-Fi client running on a Raspberry Pi. The experiment reproduced CVE-2019-9499 and the previously reported non-compliant handling of an invalid EAP version value. The results also confirmed that the reproducibility of known vulnerabilities strongly depends on the software versions installed on the target and that packet definitions for newly supported protocols must be implemented accurately according to their specifications.

We then extended Greyhound and applied it to M5Stamp Fly, an ESP32-based physical drone controlled over MQTT. We added an MQTT communication task to the drone firmware and implemented an MQTT QoS 2 state machine, application-level flight-command handling, and reboot-aware crash reporting in Greyhound. Fuzzing the physical drone revealed an unexpected UNSUBSCRIBE during the QoS 2 flow under communication delay, a specification-compliant disconnection in response to an invalid QoS value, a topic-length bounds-checking flaw that rebooted the drone, and a service failure caused by unreleased buffers in the MQTT client. We also showed that a conditional state transition based on the mutation applied to the preceding packet can prevent specification-compliant responses from being reported as false anomalies.

These results confirm that Greyhound can be extended to test drone-control systems whose communication and application behavior depend on state transitions. They also demonstrate that state-machine-guided response validation can detect and support the analysis of a broad range of problems, including protocol non-compliance, communication robustness issues, memory-safety defects, and resource-management failures, in addition to conventional crashes.

Acknowledgments We would like to thank Matheus Eduardo Garbelini, the first author of the GREYHOUND paper, for providing access to the GREYHOUND source code and for granting us permission to publicly release the original source package together with our extensions [*2]. We greatly appreciate his support, which has facilitated the development and reproducibility of this work.

[*2] `https://github.com/ertlnagoya/CSS2026_Greyhound_Artifact`

References

[1] Khandelwal, S.: Hacker Hijacks a Police Drone from 2 Km Away with $40 Kit, (online), available from ⟨https://thehackernews.com/2016/04/hacking-drone.html⟩ (accessed 2026-7-23).

[2] Wei, W.: You Can Hijack Nearly Any Drone Mid-flight Using This Tiny Gadget, (online), available from ⟨https://thehackernews.com/2016/10/how-to-hack-drone.html⟩ (accessed 2026-7-23).

[3] Schreiber, D.-P., Leithner, M., Zivanovic, J. and Simos, D. E.: Bluetooth low energy security testing with combinatorial methods, *Proceedings of the 2025 USENIX Conference on Usenix Annual Technical Conference*, USENIX ATC '25, USA, USENIX Association (2025).

[4] Garbelini, M. E., Shang, Z., Luo, S., Chattopadhyay, S., Sun, S. and Kurniawan, E.: 5Ghoul: Unleashing Chaos on 5G Edge Devices Via Stateful Multi-Layer Fuzzing, *IEEE Transactions on Dependable and Secure Computing* (2025).

[5] Shang, Z., Garbelini, M. E. and Chattopadhyay, S.: U-Fuzz: Stateful Fuzzing of IoT Protocols on COTS Devices, *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)* (2024).

[6] Shang, Z., Garbelini, M. E. and Chattopadhyay, S.: U-Fuzz: A Tool Prototype for Stateful Fuzzing of IoT Protocols on COTS Devices, *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*, pp. 448–450 (online), DOI: 10.1109/ICST60714.2024.00051 (2024).

[7] Malviya, V. K., Minn, W., Shar, L. K. and Jiang, L.: Fuzzing drones for anomaly detection: A systematic literature review, *Computers & Security*, Vol. 148, p. 104157 (online), DOI: 10.1016/j.cose.2024.104157 (2025).

[8] Kim, T., Kim, C. H., Rhee, J., Fei, F., Tu, Z., Walkup, G., Zhang, X., Deng, X. and Xu, D.: RVFuzzer: Finding Input Validation Bugs in Robotic Vehicles through Control-Guided Testing, *Proceedings of the 28th USENIX Security Symposium*, pp. 425–442 (2019).

[9] Kim, H., Ozmen, M. O., Bianchi, A., Celik, Z. B. and Xu, D.: PGFUZZ: Policy-Guided Fuzzing for Robotic Vehicles, *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, (online), DOI: 10.14722/ndss.2021.24096 (2021).

[10] Han, R., Ma, S., Li, J., Nepal, S., Lo, D., Ma, Z. and Ma, J.: Range Specification Bug Detection in Flight Control System Through Fuzzing, *IEEE Transactions on Software Engineering*, Vol. 50, No. 3, pp. 461–473 (2024).

[11] Kim, T., Kim, C. H., Ozen, A., Fei, F., Tu, Z., Zhang, X., Deng, X., Tian, D. J. and Xu, D.: From Control Model to Program: Investigating Robotic Aerial Vehicle Accidents with MAYDAY, *Proceedings of the 29th USENIX Security Symposium*, pp. 913–930 (2020).

[12] Domin, K., Marin, E. and Symeonidis, I.: Security Analysis of the Drone Communication Protocol: Fuzzing the MAVLink Protocol, *Proceedings of the 37th Symposium on Information Theory in the Benelux (SITB)* (2016).

[13] Chen, J., Diao, W., Zhao, Q., Zuo, C., Lin, Z., Wang, X., Lau, W. C., Sun, M., Yang, R. and Zhang, K.: IoTFuzzer: Discovering Memory Corruptions in IoT Through App-Based Fuzzing, *Proceedings of the Network and Distributed System Security Symposium (NDSS)* (2018).

[14] Feng, X., Sun, R., Zhu, X., Xue, M., Wen, S., Liu, D., Nepal, S. and Xiang, Y.: Snipuzz: Black-box Fuzzing of IoT Firmware via Message Snippet Inference, *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, (online), DOI: 10.1145/3460120.3484543 (2021).

[15] Pearson, B., Zhang, Y., Zou, C. and Fu, X.: FUME: Fuzzing Message Queuing Telemetry Transport Brokers, *Proceedings of the IEEE Conference on Computer Communications (INFOCOM)*, (online), DOI: 10.1109/INFOCOM48880.2022.9796755 (2022).

[16] Song, X. et al.: MBFuzzer: A Multi-Party Protocol Fuzzer for MQTT Brokers, *Proceedings of the 34th USENIX Security Symposium* (2025).

[17] : NVD - CVE-2019-9499, National Institute of Standards and Technology (online), available from ⟨https://nvd.nist.gov/vuln/detail/CVE-2019-9499⟩ (accessed 2026-6-30).

[18] Garbelini, M. E., Wang, C. and Chattopadhyay, S.: Greyhound: Directed Greybox Wi-Fi Fuzzing, *IEEE Transactions on Dependable and Secure Computing*, Vol. 19, pp. 817–834 (online), DOI: 10.1109/TDSC.2020.3014624 (2022).

[19] Espressif Systems: ESP-NOW, Document (online), available from ⟨https://www.espressif.com/en/solutions/low-power-solutions/esp-now⟩ (accessed 2026-7-23).

[20] OASIS: MQTT Version 3.1.1 – OASIS Standard, Document (online), available from ⟨https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html⟩ (accessed 2026-7-23).

© 2026 Information Processing Society of Japan